GitOpt — Frontend Design Specification

Overall design direction

Build a modern developer-focused web application with a clean, technical aesthetic.

- Dark-first interface, with optional light mode
- Inspired by GitHub, Vercel, Linear, and modern developer tooling
- Dense enough to display technical information, but not overwhelming
- Use cards, tables, charts, badges, and expandable technical details
- Avoid excessive gradients and generic AI-dashboard aesthetics
- Monospace font for Git commands, repository metrics, hashes, and technical values
- Clear visual hierarchy between **problems**, **recommendations**, and **evidence**
- Use green for healthy/optimized states, yellow/orange for warnings, and red only for significant problems
- Primary CTA should always be obvious

---

1. Landing / Home Page

Purpose

Explain what the platform does and get the developer into repository analysis as quickly as possible.

Hero section

Headline:

> **Make Git repositories faster.**

Supporting text:

> Analyze your repository, understand where Git performance is being lost, and get optimization recommendations tailored to how you work.

Primary CTA:

Analyze a Repository

Secondary CTA:

See How It Works

Visual

Show a stylized repository analysis dashboard containing:

- Repository size
- Object count
- History depth
- Clone time
- Disk usage
- Optimization recommendations

Connect these visually to recommendations such as:

Repository
↓
Analysis
↓
Detected bottleneck
↓
Recommendation
↓
Benchmark

Feature section

Three or four cards:

Repository Analysis

Analyze repository size, history, objects, files, and storage characteristics.

Workload Profiling

Tell GitOpt whether the repository is mainly used for local development, CI/CD, or monorepo development.

Actionable Recommendations

Turn repository metrics into specific Git optimization recommendations.

Benchmarking

Measure the effect of optimization strategies using real Git operations.

Bottom CTA

Stop guessing which Git optimization you need.

Button:

Analyze Your Repository

---

2. Repository Setup / Analysis Page

This is the **main entry point into the application**.

Purpose

Collect the repository and workload information needed to perform the analysis.

Repository input

Large input card:

Analyze a repository

Repository URL

https://github.com/owner/repository

Button:

Analyze Repository

Optional alternative:

Upload / Select Local Repository

Depending on implementation.

Workload selection

After entering the repository, ask:

How do you primarily use this repository?

Three selectable cards:

Local Development

Developers frequently clone the repository and work with a subset of its files and history.

CI/CD

Repositories are frequently cloned or fetched in temporary build environments.

Monorepo Development

The repository contains multiple projects or teams that typically work on different parts of the codebase.

Allow selecting one primary workload.

Could optionally allow:

+ Add another workload

Advanced options

Collapsed by default.

Possible options:

- Analyze Git history
- Analyze large objects
- Measure clone performance
- Measure checkout performance
- Measure fetch performance
- Compare optimization strategies

Button:

Start Analysis

---

3. Analysis Progress Page

Purpose

Show that the system is actually doing meaningful repository analysis rather than leaving the user staring at a spinner.

This is important because repository analysis can take time.

Header

> **Analyzing `microsoft/git`**

Status:

Analysis in progress

Progress pipeline

Display stages:

✓ Repository discovered
✓ Repository metadata collected
✓ Git objects analyzed
● History analysis
○ Workload analysis
○ Optimization evaluation

Each stage can expand to show what is happening.

For example:

> Scanning Git objects
> 1.8M objects analyzed

Live metrics

Show cards:

Repository Size

`32.4 GB`

Objects

`2.1M`

Commits

`184,293`

Files

`96,420`

Large Objects

1,284

Bottom message

> Analysis may take several minutes for large repositories. You can leave this page open while GitOpt completes the analysis.

Once complete:

View Analysis

---

4. Repository Overview / Analysis Dashboard

This is the **core page of the entire application**.

Think of it as the Git equivalent of a performance dashboard.

Header

microsoft / git

Analyzed 8 Oct 2026
Workload: Local Development

Actions:

Re-analyze

Change Workload

Overall repository health

Large summary card:

Repository Performance

Needs Optimization

Then show a simple score:

72 / 100

Don't make the score pretend to be scientific. Label it something like:

> Optimization opportunity score

Key metrics

Six cards:

Metric	Value
Repository Size	32.4 GB
Git Objects	2.1M
Commits	184K
Files	96K
Largest Object	1.8 GB
Estimated Clone	28 min

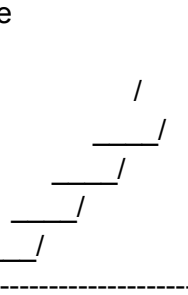
Each card should have a small explanation tooltip.

Repository structure

Visualizations could include:

Repository size over time

Line chart:



Object distribution

Donut/bar visualization:

- Blobs
- Trees
- Commits
- Tags

Largest files / objects

Table:

Object	Size	Type	Path
`model.bin`	1.8 GB	Blob	`/models/...`
`dataset.zip`	940 MB	Blob	`/data/...`

Detected issues

This section is extremely important.

Cards such as:

Large Repository History

High impact

> The repository contains a large amount of historical data that may not be required for your selected local-development workflow.

Potential optimization

`Partial Clone`

Expected benefit

Reduced initial data transfer and faster repository setup.

Button:

View Recommendation

Deep History

Medium impact

> Your workload may not require the complete commit history during initial setup.

Potential optimization

`Shallow Clone`

Button:

View Recommendation

---

5. Recommendations Page

This should be the **most useful page**, not just another analytics dashboard.

Header

> **Optimization Recommendations**

Subtitle:

> Based on your repository characteristics and **Local Development** workload.

Recommendation cards

Each recommendation gets a card.

01 — Use Partial Clone

High impact

Why this was recommended

Your repository contains a large amount of Git object data, while your selected workload does not necessarily require all file contents immediately.

Potential benefit

Initial download
32.4 GB → estimated 6.8 GB

Recommended command

```
git clone --filter=blob:none \
  https://github.com/owner/repository.git
```

Button:

Copy Command

Secondary:

View Details

---

02 — Use Sparse Checkout

Medium impact

> Only a subset of the repository appears relevant to the selected workload.

Command:

```
git sparse-checkout init --cone
git sparse-checkout set src/backend
```

Buttons:

Copy Configuration

View Details

---

03 — Consider Shallow Clone

Low/Medium impact

Explain why it helps and when it **doesn't**.

This distinction matters.

The application shouldn't blindly recommend every optimization.

Recommendation confidence

Each recommendation could display:

Confidence: High

with an expandable explanation:

> Based on repository size, object distribution, history depth, and measured clone behavior.

This makes the recommendations feel evidence-based rather than AI-generated nonsense.

---

6. Recommendation Detail Page

Clicking a recommendation opens a detailed technical explanation.

Header

> Partial Clone

Recommended for Local Development

Problem

> Your repository contains a large amount of blob data that contributes significantly to clone size.

Why it matters

Explain the Git mechanism in relatively simple language.

Could include a diagram:

Traditional Clone

Repository
↓
All commits
All trees
All blobs
↓
Developer

versus:

Partial Clone

Repository
↓
Commits + Trees
↓
Developer
↓
Fetch blob when required

Recommended configuration

Large code block:

```
git clone --filter=blob:none \
  https://github.com/owner/repository.git
```

Button:

Copy Command

Trade-offs

This is important academically and technically.

Advantages

- Smaller initial download
- Faster initial setup
- Lower disk usage

Trade-offs

- Missing objects may need to be fetched later
- Performance depends on network availability
- Some workflows may not benefit

Expected impact

Show:

Metric	Current	Optimized
Clone size	32.4 GB	6.8 GB
Clone time	28 min	8 min

Disk usage	34 GB	9 GB

Then clearly label these as:

Measured

or

Estimated

Never mix the two.

---

7. Benchmark Page

This is where the project becomes much more interesting than a repository analyzer.

Purpose

Actually compare Git strategies.

Header

> **Optimization Benchmark**

Repository:

`microsoft/git`

Workload:

`Local Development`

Strategy comparison

Large comparison table:

Strategy	Clone Time	Data Transferred	Disk Usage
Full Clone	28m 14s	32.4 GB	34.1 GB

Shallow Clone	12m 08s	14.2 GB	15.3 GB
Partial Clone	**8m 21s**	**6.8 GB**	**9.1 GB**
Partial + Sparse	**6m 43s**	**5.2 GB**	**6.7 GB**

Highlight the best result.

Performance charts

Clone time

Bar chart comparing strategies.

Data transferred

Bar chart.

Disk usage

Bar chart.

Recommendation

At the bottom:

> **Best strategy for this workload: Partial Clone + Sparse Checkout**

Then:

Apply Recommendation

or

View Configuration

---

8. Configuration / Export Page

Purpose

Turn recommendations into something the developer can actually use.

Header:

Your Git Configuration

Tabs:

Commands | Config | Documentation

Example:

```
# Clone repository without downloading all blob contents
git clone --filter=blob:none \
  https://github.com/example/repository.git

# Enable sparse checkout
git sparse-checkout init --cone

# Select required directory
git sparse-checkout set src/backend
```

Buttons:

Copy All

Download Configuration

Potentially generate a Markdown report:

```
GitOpt Analysis Report
Repository: example/repository
Workload: Local Development

Recommendations:
1. Partial Clone
2. Sparse Checkout

Expected improvement:
...
```

---

9. Analysis History Page

Optional but useful even for the MVP.

Purpose

Allow developers to see previous analyses.

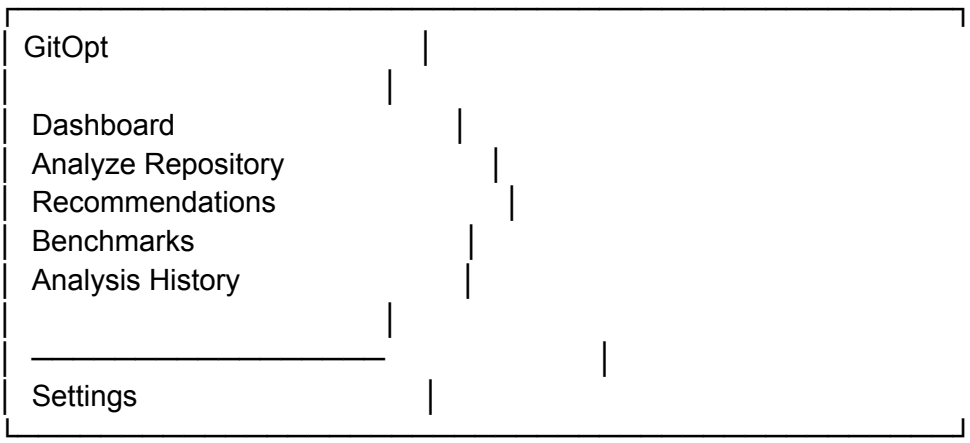
Table:

Repository	Workload	Score	Analyzed	
`microsoft/git`	Local Dev	72	Oct 8	View
`company/monorepo`	CI/CD	84	Oct 4	View
`app/project`	Monorepo	61	Sep 28	View

This also gives the application somewhere to go after the first analysis.

Recommended Navigation

Keep the navigation extremely simple:

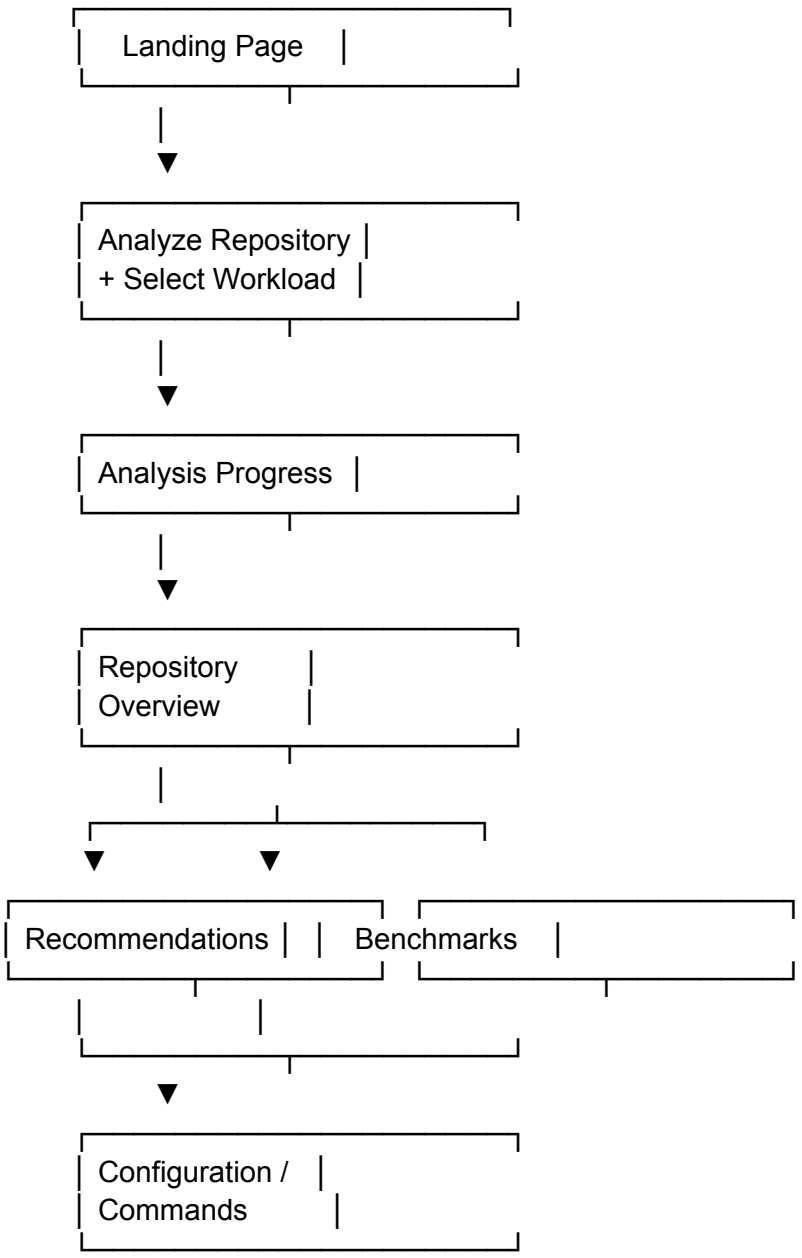


For an individual repository, the navigation could become:

Repository
├── Overview
├── Metrics
├── Recommendations
├── Benchmarks
└── Configuration

The Core User Journey

I'd tell Figma AI to design the whole application around this flow:



One important design decision

Don't make this an "AI recommends Git commands" app.

That's weak.

The interesting part of the project is the chain:

> **Repository characteristics → workload → detected bottleneck → Git mechanism → measurable improvement**

The UI should make that chain visible everywhere.

For example:

Repository has 32 GB of blob data
↓
Developer only needs a subset during local development
↓
Partial clone is appropriate
↓
Clone drops from 28 min → 8 min
↓
Here's the exact command